

Predicting Customer Churn in the Telecommunications Industry: A Comparative Study of Classification Algorithms

BIL 476 — Data Mining Course Project, Summer 2026

TOBB University of Economics and Technology

Zeynep Ay

TOBB University of Economics and Technology

Ankara, Turkey

Student ID: 231404035

zay@etu.edu.tr

Abstract— Customer churn is a costly problem for telecommunications companies because replacing customers generally requires more resources than retaining them. This project predicts churn from account, service, and billing attributes using the public IBM Telco Customer Churn dataset. The dataset contains 7,043 customer records, 19 predictive attributes of mixed nominal and numeric types, and a binary churn target. Data cleaning revealed 11 blank values hidden in the `TotalCharges` field and a class imbalance of 73.5% retained customers versus 26.5% churned customers. Four classifiers—Decision Tree, Gaussian Naive Bayes, k-Nearest Neighbors, and Random Forest—were tuned with stratified five-fold cross-validation and evaluated on an untouched 20% test set. Preprocessing was fitted within each cross-validation fold to prevent leakage. Baseline models were compared with variants using SMOTENC, which respects the dataset’s mixed categorical and numeric feature types. The best baseline accuracy was 80.3%, achieved by Random Forest, which also produced the highest ROC-AUC (0.839) and PR-AUC (0.653). Random Forest with SMOTENC achieved the highest F1-score (0.605), with 67.4% recall and 54.9% precision. Naive Bayes with SMOTENC produced a similar F1-score (0.603) with higher recall (77.8%) but lower precision (49.2%). These results show that the preferred model depends on whether a retention campaign prioritizes detecting more churners or limiting false alarms.

Index Terms— data mining, customer churn, classification, telecommunications, class imbalance

1 INTRODUCTION

Customer churn occurs when a customer stops using a company’s service. It is an important problem for telecommunications companies because attracting a new customer generally costs more than keeping an existing one [2]. If customers who are likely to leave can be identified early, a company can direct retention offers to those customers instead of applying the same campaign to everyone.

This project asks how accurately customer churn can be predicted from account, service, and billing information, and how different classifiers behave when the churn class is smaller than the non-churn class. We compare Decision Tree, Gaussian Naive Bayes, k-Nearest Neighbors (k-NN), and Random Forest on the IBM Telco Customer Churn dataset. The algorithms use different learning strategies, allowing their predictive performance and practical trade-offs to be compared.

The remainder of this paper is organized as follows. Section 2 reviews related work on customer churn prediction. Section 3 describes the dataset and its characteristics. Section 4 details the data preprocessing pipeline, the algorithms used, and the experimental setup. Section 5 presents the experimental re-

sults. Section 6 discusses the findings, limitations, and ethical considerations. Section 7 concludes the paper and outlines directions for future work.

2 RELATED WORK

Predicting customer churn in telecommunications has been studied with many classification methods. Ahmad et al. [1] used data from a Syrian telecom operator and compared Decision Tree, Random Forest, Gradient Boosted Machine, and XGBoost. XGBoost gave their strongest result, and the study also emphasized the low proportion of churn customers. Mustafa et al. [2] compared six classifiers on data from a Malaysian telecom operator; among their tested methods, CART achieved the highest accuracy. Sana et al. [3] evaluated eight algorithms with several data-transformation methods across four public telecom datasets and found improvements from combining transformations with ensemble classifiers.

Wagh et al. [5] also compared multiple machine-learning classifiers for telecom churn and reported strong performance from ensemble tree methods. Chang et al. [6] combined churn classifiers with SHAP and LIME to examine the variables influencing predictions. These studies show that model comparison, class imbalance, and interpretability are recurring concerns in churn prediction. Chawla et al. [4] introduced SMOTE for minority-class oversampling. This project uses SMOTENC, its mixed-data extension, so categorical variables are handled as categories during sample generation.

This project has a narrower objective. Instead of trying to obtain the highest possible score with many transformations or advanced models, it provides a reproducible baseline comparison of four common classifiers on one public dataset. Each classifier is evaluated both with the original training distribution and with SMOTENC, making the effect of imbalance handling easier to observe.

3 DATASET DESCRIPTION

3.1 Data Source

The IBM Telco Customer Churn dataset was originally released as IBM sample data and is available on Kaggle [7, 8]. The verified Kaggle page is <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>. It represents a fictional telecommunications company and records each customer’s demographic information, subscribed services, contract and billing information, and whether the customer churned. The dataset was selected because it represents a clear classification problem, contains both nominal and numeric at-

tributes, and is small enough to reproduce on a standard computer.

3.2 Data Characteristics

Table 1 summarizes the key properties of the dataset.

Table 1: Dataset Summary

Property	Value
Number of Records	7,043
Number of Predictive Attributes	19 (customer ID excluded)
Attribute Types	16 Nominal, 3 Numeric
Target Variable	Churn (Yes / No)
Missing / Invalid Values	11 records (0.16%) with a blank <code>TotalCharges</code> value (all with <code>tenure = 0</code>); corrected during preprocessing (Section 4)
Class Distribution	73.5% No-Churn / 26.5% Churn
Source URL	kaggle.com/datasets/blastchar/telco-customer-churn

The three numeric attributes are `tenure` (number of months the customer has stayed with the company), `MonthlyCharges`, and `TotalCharges`. Table 2 reports descriptive statistics for these attributes computed on the cleaned dataset.

Table 2: Descriptive Statistics for Numeric Attributes

Attribute	Mean	Std.	Min	Max
<code>tenure</code> (months)	32.37	24.56	0	72
<code>MonthlyCharges</code> (\$)	64.76	30.09	18.25	118.75
<code>TotalCharges</code> (\$)	2279.73	2266.79	0.00	8684.80

The 16 nominal attributes include demographic fields (`gender`, `SeniorCitizen`, `Partner`, `Dependents`), service-subscription fields (`PhoneService`, `MultipleLines`, `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`), and account fields (`Contract`, `PaperlessBilling`, `PaymentMethod`).

3.3 Ethical Considerations

The dataset is public sample data and does not contain customer names or addresses. Its `customerID` field is an anonymized identifier and was removed before modeling. However, the dataset includes demographic attributes such as `gender` and `SeniorCitizen`. If a churn model were used in practice, its errors should therefore be compared across customer groups before it is used to guide retention decisions. Feature importance alone cannot prove that a model is fair. Both ChatGPT and Claude were used during this project for coding, debugging, and pipeline improvement. The student prepared the initial report outline, section structure, and content plan; ChatGPT then assisted with developing parts of the report, language revision, structural refinement, and LaTeX formatting.

Only this public dataset was used; no private or confidential customer data was shared with these AI tools.

4 METHODOLOGY

4.1 Data Preprocessing

The following preprocessing steps were applied to the raw dataset:

- **Missing/invalid value handling:** Although a naive null check reports zero missing values, 11 records store an empty string in the `TotalCharges` field instead of a numeric value. These records were identified by converting the column to a numeric type and inspecting the resulting missing values. All 11 correspond to customers with `tenure = 0`, i.e. brand-new customers who have not yet been billed; these values were therefore imputed as 0 rather than with a summary statistic such as the mean, since 0 is the value that is actually consistent with these customers’ billing history.
- **Identifier removal:** The `customerID` column was removed, since it is a unique identifier with no predictive value and its inclusion would risk data leakage or spurious memorization.
- **Consistency recoding:** The `SeniorCitizen` attribute, originally encoded as 0/1, was recoded to Yes/No to be consistent with the representation of the dataset’s other binary nominal attributes.
- **Target encoding:** The target variable `Churn` (Yes/No) was additionally encoded as a binary indicator (`Churn_binary`, 1 = Yes) to support both the descriptive analysis in Section 3 and downstream model training.

Categorical attributes were one-hot encoded, while the three continuous attributes (`tenure`, `MonthlyCharges`, and `TotalCharges`) were standardized for k-NN because distance-based models are sensitive to feature scale. Encoding and scaling were implemented as pipeline steps fitted separately within each cross-validation training fold. This prevents validation-fold statistics from leaking into model training. Decision Tree, Naive Bayes, and Random Forest received unscaled numeric features. To evaluate class-imbalance handling, each baseline model was compared with a SMOTENC variant. SMOTENC was also placed inside the cross-validation pipeline, so synthetic samples were generated only from each training fold and categorical attributes were handled explicitly as categorical rather than interpolated as one-hot values.

4.2 Algorithm Selection and Description

Four classification algorithms were selected to compare a range of modeling assumptions, from simple and interpretable to ensemble-based:

- **Decision Tree:** A recursive, rule-based classifier that splits the feature space based on information-theoretic criteria such as Gini impurity or entropy. It was selected for its interpretability: the resulting tree structure can be read

directly by a non-technical stakeholder as a set of “if-then” retention rules.

- **Naive Bayes:** A probabilistic classifier based on Bayes’ theorem under a (naive) conditional independence assumption between features. It was selected as a computationally inexpensive baseline that is known to perform competitively on datasets with largely categorical attributes, as in [2].
- **k-Nearest Neighbors (k-NN):** An instance-based, non-parametric classifier that assigns the majority class among the k nearest training examples. It was selected because it can capture local patterns in the preprocessed feature space.
- **Random Forest:** An ensemble of decision trees trained on bootstrap samples with random feature subsets, combined via majority voting. It was selected because ensemble methods have consistently outperformed single classifiers in the churn-prediction literature reviewed in Section 2 [1, 3], while still allowing feature importance to be extracted for interpretability.

Together, these algorithms allow models with different levels of complexity and interpretability to be compared.

4.3 Experimental Setup

- **Train/test split:** 80% / 20% stratified split, preserving the 73.5%/26.5% class ratio in both subsets.
- **Hyperparameter selection:** Key hyperparameters (e.g., tree depth for Decision Tree and Random Forest and k for k-NN) were selected by stratified 5-fold cross-validation on the training set, optimizing churn-class F1-score. Baseline and SMOTENC variants were tuned separately.
- **Software environment:** Python 3.10, with pandas and NumPy for data handling, scikit-learn and imbalanced-learn for model training and evaluation, and matplotlib/seaborn for visualization.
- **Hardware:** Standard consumer-grade hardware (CPU only); no specialized hardware is required given the dataset size.

4.4 Reproducibility and Verification

All preprocessing and modeling steps are implemented in Jupyter notebooks. A fixed random seed (`random_state=42`) is used for the train/test split, cross-validation, SMOTENC, and stochastic classifiers. The GitHub repository contains the notebooks, dependency list, dataset link, generated results, and execution instructions. The notebooks were run in the stated order and the reported tables were checked against the exported results file.

5 RESULTS

All four algorithms were trained both on the original imbalanced training data and with SMOTENC inside their cross-validation pipelines. The selected models were then evaluated

once on the same held-out, untouched 20% test set (1,409 customers), which retained the real-world 73.5%/26.5% class ratio.

5.1 Evaluation Metrics

Because the target class is imbalanced (73.5%/26.5%), accuracy alone would be misleading: a trivial classifier that always predicts “No Churn” would achieve 73.5% accuracy while never identifying a churner. We therefore report Accuracy, Precision, Recall, F1-score, ROC-AUC, and PR-AUC for each model. PR-AUC is particularly informative here because it focuses on the precision–recall trade-off for the minority class. Recall and F1-score are emphasized because failing to identify a customer who will churn may be more costly to a retention program than incorrectly flagging a loyal customer.

5.2 Results Summary

Table 3 reports test-set performance for all eight model variants (four baseline and four SMOTENC-trained), sorted by F1-score.

Table 3: Test-Set Performance, Sorted by F1-score

Model	Acc.	Prec.	Rec.	F1	ROC	PR
Random Forest (SMOTENC)	0.767	0.549	0.674	0.605	0.830	0.616
Naive Bayes (SMOTENC)	0.728	0.492	0.778	0.603	0.804	0.575
k-NN (SMOTENC)	0.730	0.495	0.762	0.600	0.808	0.560
Decision Tree	0.798	0.635	0.567	0.599	0.828	0.621
Naive Bayes	0.703	0.465	0.810	0.591	0.819	0.603
Decision Tree (SMOTENC)	0.728	0.492	0.738	0.590	0.791	0.499
Random Forest	0.803	0.662	0.524	0.585	0.839	0.653
k-NN	0.780	0.589	0.564	0.577	0.827	0.601

5.3 Visualizations

Figures 1–5 provide complementary views of the results. They compare F1-scores, ROC curves, precision–recall curves, and confusion matrices for the evaluated variants. The final figure reports the feature importance values produced by the baseline Random Forest model.

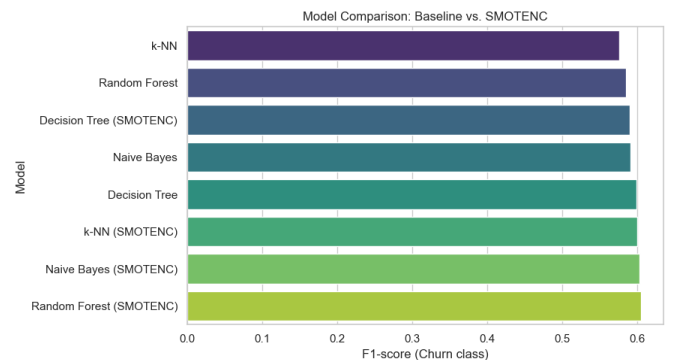


Figure 1: F1-score comparison across all eight model variants.

5.4 Comparison of Approaches

The baseline Random Forest achieved the highest accuracy (80.3%), ROC-AUC (0.839), and PR-AUC (0.653). Its precision was also the highest (66.2%), but its churn recall was only

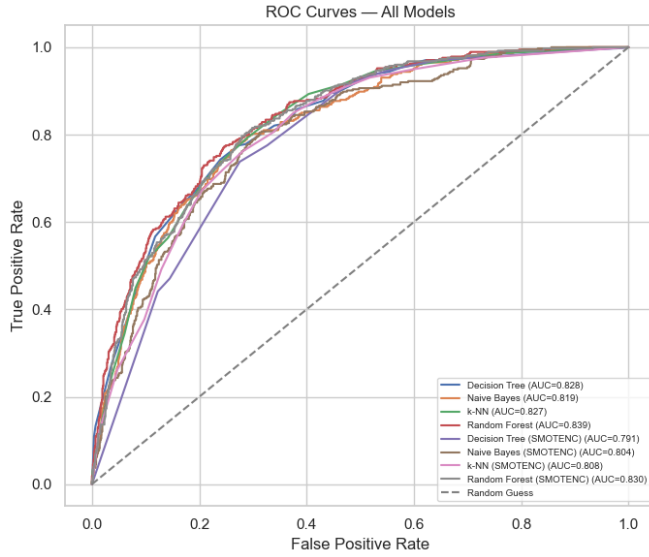


Figure 2: ROC curves for all eight baseline and SMOTENC-trained model variants.

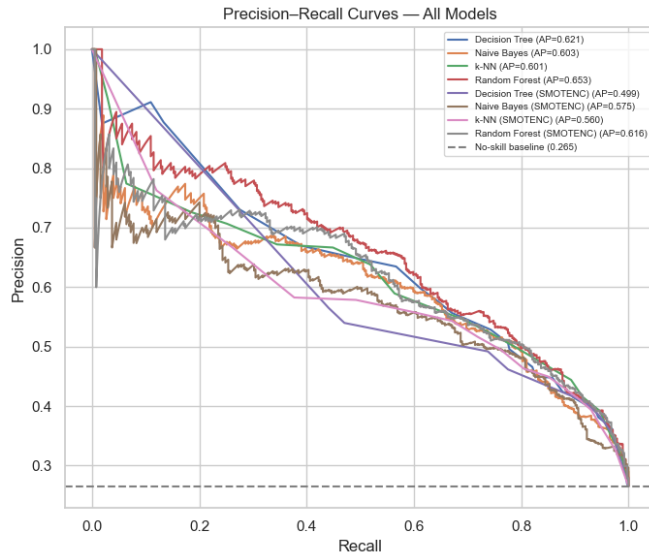


Figure 3: Precision–recall curves for all eight model variants.

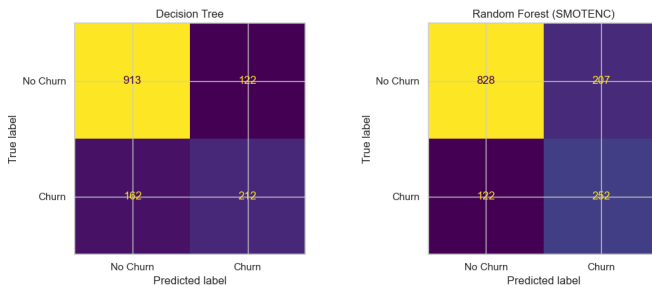


Figure 4: Confusion matrices for the best baseline model (Decision Tree, by F1) and the best SMOTENC model (Random Forest, by F1).

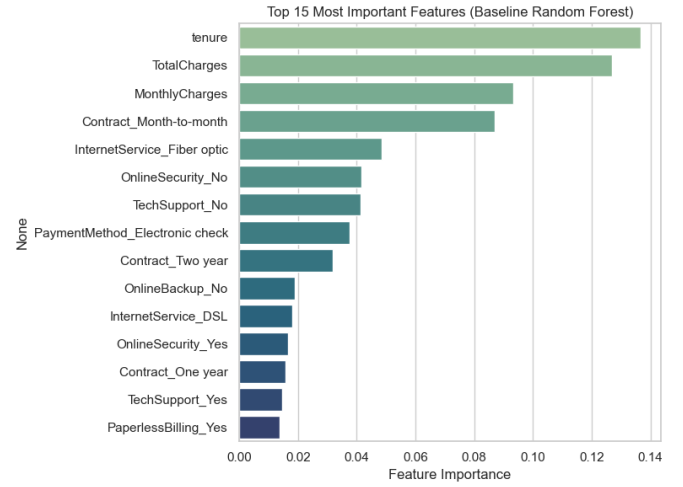


Figure 5: Top 15 features by importance, from the baseline Random Forest model.

52.4%, so it missed almost half of the actual churners. Baseline Naive Bayes showed the opposite behavior: it achieved the highest recall (81.0%) but only 46.5% precision. The baseline Decision Tree produced the highest baseline F1-score (0.599), balancing 63.5% precision with 56.7% recall.

SMOTENC increased recall for Decision Tree, k-NN, and Random Forest, while Naive Bayes already had very high baseline recall and decreased slightly from 81.0% to 77.8%. In each algorithm family, the SMOTENC variant traded some accuracy or precision for higher minority-class sensitivity. Random Forest with SMOTENC achieved the highest F1-score (0.605), while Naive Bayes with SMOTENC was close at 0.603. Random Forest with SMOTENC offered higher precision (54.9% versus 49.2%), whereas Naive Bayes with SMOTENC offered higher recall (77.8% versus 67.4%). The small F1 difference means that a practical selection should be based on retention costs rather than ranking alone: Naive Bayes with SMOTENC favors catching more churners, whereas Random Forest with SMOTENC produces fewer false alarms. If probability ranking or minimizing unnecessary interventions is more important, the baseline Random Forest remains the strongest option because of its superior ROC-AUC, PR-AUC, and precision.

6 DISCUSSION

Best-performing method. No single algorithm dominated every metric. Random Forest’s ensemble reduced the variance of a single Decision Tree and produced the strongest baseline ranking performance, as shown by both ROC-AUC and PR-AUC. However, using the default 0.5 decision threshold, it predicted the minority class conservatively and therefore had modest churn recall. SMOTENC shifted the Random Forest toward higher recall, although precision declined. Random Forest with SMOTENC achieved the highest F1-score, but its lead over Naive Bayes with SMOTENC was only 0.002. This difference is too small to justify a deployment decision by itself. A retention team should instead choose between these models using the relative business cost of a missed churner and an unnecessary offer.

Limitations. This study is based on a single dataset from one fictional telecom provider, so the results may not generalize to companies with different customer bases, pricing structures, or markets. Hyperparameter search was limited to small grids for tractability rather than an exhaustive or nested search. The final test set was used only after model selection, but a separate temporal or external validation set was unavailable, so performance on genuinely future customers remains unknown. SMOTENC avoids invalid interpolation across categorical codes, but its synthetic minority examples still depend on local patterns in the observed churners and may not represent all future churn behavior. Finally, the models were compared at the default 0.5 threshold rather than thresholds optimized for a known business cost.

Observed model behavior. Naive Bayes obtained high recall even though its conditional-independence assumption is not realistic for all attributes in this dataset. For example, several internet-service fields are related to one another. This result shows that a model with a simple assumption can still identify a large proportion of churn customers, although its lower precision means that it also produces more false alarms.

Effect of data-quality issues. The hidden `TotalCharges` issue occurred in only 11 of 7,043 records (0.16%), so its effect was probably limited; however, this was not tested separately. It is still important because a standard null count did not reveal the blank strings. Class imbalance had a clearer effect on evaluation, as shown by the accuracy–recall trade-off between the baseline and SMOTENC variants.

Ethical implications. Demographic attributes were not among the most important Random Forest features in Fig. 5, but this does not establish fairness. Before real use, performance and error rates should be checked across relevant customer groups.

7 CONCLUSION AND FUTURE WORK

This project compared Decision Tree, Gaussian Naive Bayes, k-NN, and Random Forest classifiers for predicting customer churn, with both baseline and SMOTENC training pipelines. The baseline Random Forest achieved the highest accuracy (80.3%), precision (66.2%), ROC-AUC (0.839), and PR-AUC (0.653). Random Forest with SMOTENC achieved the highest F1-score (0.605), with 67.4% recall and 54.9% precision. Naive Bayes with SMOTENC achieved a similar F1-score (0.603) and higher recall (77.8%), but lower precision (49.2%). The main practical conclusion is therefore not that one model is universally best. A high-recall retention campaign could prefer Naive Bayes with SMOTENC, while a campaign seeking to reduce unnecessary offers or rank customers reliably could prefer Random Forest. The Random Forest feature-importance analysis identifies which variables most influenced its predictions, but it does not establish the direction of those effects or a causal relationship with churn.

Future work could compare additional algorithms such as Logistic Regression, Gradient Boosting, and XGBoost [1]; test class weighting as an alternative to SMOTENC; and select decision thresholds using the actual costs of false positives and false negatives. Testing the models on data from another provider or a later time period would also show whether the

results generalize.

AI TOOL USE AND ORIGINALITY DECLARATION

Both ChatGPT and Claude were used for coding and debugging support and for improving the experimental pipeline. The student prepared the initial report outline, section structure, and content plan. ChatGPT assisted with developing parts of the report, locating literature candidates, language revision, structural refinement, and LaTeX formatting. The notebooks were executed, the numerical results were checked against the exported output, and the final wording, tables, figures, and citations were reviewed, verified, and revised by the student. The student accepts full responsibility for the accuracy, originality, reproducibility, and integrity of the submitted work.

REFERENCES

- [1] A. K. Ahmad, A. Jafar, and K. Aljoumaa, “Customer churn prediction in telecom using machine learning in big data platform,” *Journal of Big Data*, vol. 6, no. 1, p. 28, 2019.
- [2] N. Mustafa, L. Sook Ling, and S. F. Abdul Razak, “Customer churn prediction for telecommunication industry: A Malaysian case study,” *F1000Research*, vol. 10, p. 1274, 2021.
- [3] J. K. Sana, M. Z. Abedin, M. S. Rahman, and M. S. Rahman, “A novel customer churn prediction model for the telecommunication industry using data transformation methods and feature selection,” *PLOS ONE*, vol. 17, no. 12, p. e0278095, 2022.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [5] S. K. Wagh, A. A. Andhale, K. S. Wagh, J. R. Pansare, S. P. Ambadekar, and S. H. Gawande, “Customer churn prediction in telecom sector using machine learning techniques,” *Results in Control and Optimization*, vol. 14, p. 100342, 2024.
- [6] V. Chang, K. Hall, Q. A. Xu, F. O. Amao, M. A. Ganatra, and V. Benson, “Prediction of customer churn behavior in the telecommunication industry using machine learning models,” *Algorithms*, vol. 17, no. 6, p. 231, 2024.
- [7] IBM, “Telco customer churn sample data,” *IBM Cognos Analytics Documentation*. [Online]. Available: <https://www.ibm.com/docs/en/cognos-analytics/12.0.x?topic=samples-telco-customer-churn>. Accessed: Jul. 28, 2026.
- [8] BlastChar, “Telco Customer Churn,” *Kaggle*. [Online]. Available: <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>. Accessed: Jul. 28, 2026.

A CODE REPOSITORY

The source code, Jupyter notebooks, dependency list, results, and execution instructions are available at:

<https://github.com/Bil-476-Project/Bil-476-Project>

B ADDITIONAL FIGURES AND TABLES

The main results table, F1-score comparison, ROC curves, precision–recall curves, confusion matrices, and feature-importance plot are included in Section 5. Table 4 reports the best hyperparameters selected via stratified 5-fold cross-validation (optimizing F1-score). Baseline and SMOTENC variants were tuned independently.

Table 4: Selected Hyperparameters (5-fold CV, optimizing F1)

Model	Best Hyperparameters	CV F1
Decision Tree	max_depth=5, min_samples_leaf=10	0.565
Naive Bayes	var_smoothing=1e-7	0.599
k-NN	n_neighbors=21	0.591
Random Forest	n_estimators=200, max_depth=10	0.578
Decision Tree (SMOTENC)	max_depth=3, min_samples_leaf=1	0.593
Naive Bayes (SMOTENC)	var_smoothing=1e-9	0.612
k-NN (SMOTENC)	n_neighbors=15	0.608
Random Forest (SMOTENC)	n_estimators=200, max_depth=10	0.626